

pandas 데이터 파악과 조작

분석할 데이터를 수집(확보)하면 데이터의 특징을 파악하고 다루기 쉽게 변형하는 작업을 수행해야 한다

#2. 데이터 조작(가공)

- 데이터 개수 세기 : `count()`, `value_counts()`
- 데이터 정렬 : `sort_values()`, `sort_index()`
- 데이터 순위 : `rank()`
- 데이터 집계 : 합계(`sum()`), 평균(`mean()`), 최대(`max()`), 최소(`min()`)
- 데이터 삭제 : `drop(axis=0/1)`
- 결측치 처리 : `dropna(axis=0/1, subset, inplace)`
- 중복값 처리 : `duplicated()`, `drop_duplicates()`
- 데이터 변경 :
 - 단위환산
 - 자료형 변경 : `astype()`
 - 수치형 데이터를 범주형 데이터로 변경 :
 - 구간을 지정하여 범주화 : `cut(data, bins, labels)`
 - 동일한 개수를 갖도록 범주화 : `qcut(data, bins_num, labels)`
 - 스케일링(`scaling`) : 정규화(`min-max scaling`), 표준화(`z-score`)
 - 시계열데이터 처리
- 행/열에 동일한 함수 적용 : `apply()`, `applymap()`, `map()`
- 열 재구성 : `DataFrame.str.split()`, `Series.str.get()`
- 필터링 : `isin()`

```
In [1]: import numpy as np
import pandas as pd
print(f'Numpy:{np.__version__}, Pandas:{pd.__version__}')
```

Numpy:2.5.2, Pandas:3.0.5

```
In [2]: from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
```

8. 열 재구성

열 순서 변경

- 데이터프레임의 열 순서 변경
- 형식 : `df[재구성한 열이름리스트]`

```
In [3]: import seaborn as sns
sns.__version__
```

Out[3]: '0.13.2'

```
In [4]: titanic = sns.load_dataset('titanic')
titanic.head()
```

Out[4]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN



```
In [5]: df = titanic.loc[0:4, 'survived':'age']
df
```

Out[5]:

	survived	pclass	sex	age
0	0	3	male	22.0
1	1	1	female	38.0
2	1	3	female	26.0
3	1	1	female	35.0
4	0	3	male	35.0

```
In [6]: col_names = list(df.columns)
col_sorted = sorted(col_names)
col_sorted
```

Out[6]: ['age', 'pclass', 'sex', 'survived']

```
In [8]: # 열 재구성
df[col_sorted]
```

Out[8]:

	age	pclass	sex	survived
0	22.0	3	male	0
1	38.0	1	female	1
2	26.0	3	female	1
3	35.0	1	female	1
4	35.0	3	male	0

```
In [11]: col_names
col_rev = list(reversed(col_names))
df[col_rev]
```

Out[11]: ['survived', 'pclass', 'sex', 'age']

```
Out[11]:
```

	age	sex	pclass	survived
0	22.0	male	3	0
1	38.0	female	1	1
2	26.0	female	3	1
3	35.0	female	1	1
4	35.0	male	3	0

열 분리

```
In [13]: # 넷플릭스 주가 데이터
# https://www.kaggle.com/datasets/pritheta/netflix-stock-data-from-2002-to-2021?resource=download
df = pd.read_csv('data/NFLX.csv')
df.head()
df.info()
df.shape
```

```
Out[13]:
```

	Date	Open	High	Low	Close	Adj Close	Volume
0	2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000
1	2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800
2	2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400
3	2002-05-29	1.164286	1.164286	1.085714	1.103571	1.103571	6757800
4	2002-05-30	1.107857	1.107857	1.071429	1.071429	1.071429	10154200

```
<class 'pandas.DataFrame'>
RangeIndex: 4874 entries, 0 to 4873
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        4874 non-null   str
1   Open        4874 non-null   float64
2   High        4874 non-null   float64
3   Low         4874 non-null   float64
4   Close       4874 non-null   float64
5   Adj Close   4874 non-null   float64
6   Volume      4874 non-null   int64
dtypes: float64(5), int64(1), str(1)
memory usage: 266.7 KB
```

```
Out[13]: (4874, 7)
```

Date 컬럼의 연-월-일을 ['Year','Month','Day']로 분리

```
In [16]: dates = df['Date'].str.split('-')
# df['Year'] = dates.str[0]
# df['Month'] = dates.str[1]
# df['Day'] = dates.str[-1]
```

시리즈의 문자열 리스트 인덱싱 : Series.str.get(인덱스)

```
In [17]: dates.str.get(0)
```

```
Out[17]: 0      2002
1      2002
2      2002
3      2002
4      2002
...
4869   2021
4870   2021
4871   2021
4872   2021
4873   2021
```

Name: Date, Length: 4874, dtype: object

['Year','Month','Day']로 분리된 시리즈문자열 요소를 데이터프레임 열로 추가

```
In [20]: # df['Year'] = dates.str.get(0)
# df['Month'] = dates.str.get(1)
# df['Day'] = dates.str.get(2)

# apply를 이용한 분리
new_df = dates.apply(pd.Series)
new_df.columns = ['Year', 'Month', 'Day']
new_df.index = df.index
df = df.join(new_df)
df.head(3)
```

```
Out[20]:
```

	Date	Open	High	Low	Close	Adj Close	Volume	Year	Month	Day
0	2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000	2002	05	23
1	2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800	2002	05	24
2	2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400	2002	05	28

문제. Date컬럼을 날짜형식 데이터로 변환한 후 연, 월, 일 컬럼을 추가

```
In [22]: # df['Date_'] = df['Date'].astype('datetime64[ns]')
df['Date_'] = pd.to_datetime(df['Date'], format='%Y-%m-%d', errors='coerce')
df.head()
```

```
Out[22]:
```

	Date	Open	High	Low	Close	Adj Close	Volume	Year	Month	Day	Date_
0	2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000	2002	05	23	2002-05-23
1	2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800	2002	05	24	2002-05-24
2	2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400	2002	05	28	2002-05-28
3	2002-05-29	1.164286	1.164286	1.085714	1.103571	1.103571	6757800	2002	05	29	2002-05-29
4	2002-05-30	1.107857	1.107857	1.071429	1.071429	1.071429	10154200	2002	05	30	2002-05-30

```
In [23]: df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 4874 entries, 0 to 4873
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        4874 non-null   str
1   Open        4874 non-null   float64
2   High        4874 non-null   float64
3   Low         4874 non-null   float64
4   Close       4874 non-null   float64
5   Adj Close   4874 non-null   float64
6   Volume      4874 non-null   int64
7   Year        4874 non-null   str
8   Month       4874 non-null   str
9   Day         4874 non-null   str
10  Date_       4874 non-null   datetime64[us]
dtypes: datetime64[us](1), float64(5), int64(1), str(4)
memory usage: 419.0 KB

```

```

In [24]: df['연'] = df['Date_'].dt.year
df['월'] = df['Date_'].dt.month
df['일'] = df['Date_'].dt.day
df['요일'] = df['Date_'].dt.dayofweek
df.head()

```

```

Out[24]:

```

	Date	Open	High	Low	Close	Adj Close	Volume	Year	Month	Day	Date_	
0	2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000	2002	05	23	2002-05-23	2002-05-23
1	2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800	2002	05	24	2002-05-24	2002-05-24
2	2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400	2002	05	28	2002-05-28	2002-05-28
3	2002-05-29	1.164286	1.164286	1.085714	1.103571	1.103571	6757800	2002	05	29	2002-05-29	2002-05-29
4	2002-05-30	1.107857	1.107857	1.071429	1.071429	1.071429	10154200	2002	05	30	2002-05-30	2002-05-30

```

In [26]: df['요일_'] = df['요일'].map({0:'월',1:'화',2:'수',3:'목',4:'금',5:'토',6:'일'})
df.head()

```

Out[26]:

	Date	Open	High	Low	Close	Adj Close	Volume	Year	Month	Day	Date_	
0	2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000	2002	05	23	2002-05-23	2002-05-23
1	2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800	2002	05	24	2002-05-24	2002-05-24
2	2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400	2002	05	28	2002-05-28	2002-05-28
3	2002-05-29	1.164286	1.164286	1.085714	1.103571	1.103571	6757800	2002	05	29	2002-05-29	2002-05-29
4	2002-05-30	1.107857	1.107857	1.071429	1.071429	1.071429	10154200	2002	05	30	2002-05-30	2002-05-30



- .dt 접근자 : 반드시 datetime 타입에서만 사용 가능
- 대용량데이터는 to_datetime() 함수 사용시 'format=' 지정이 유리(astype사용한 경우보다 속도 빠를 수 있음)
- astype()을 이용한 날짜형식 변경 시 속도는 빠르나 에러 처리 옵션이 없음

9. 필터링

(1) 불린 인덱싱(boolean indexing)

In [27]: `titanic.tail()`

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
886	0	2	male	27.0	0	0	13.00	S	Second	man	True	Na
887	1	1	female	19.0	0	0	30.00	S	First	woman	False	
888	0	3	female	NaN	1	2	23.45	S	Third	woman	False	Na
889	1	1	male	26.0	0	0	30.00	C	First	man	True	
890	0	3	male	32.0	0	0	7.75	Q	Third	man	True	Na



나이가 10대(10~19세) 승객 추출

In [28]: `mask = (titanic.age>=10) & (titanic.age <=19)`
`titanic.loc[mask,:]`

Out[28]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male
9	1	2	female	14.0	1	0	30.0708	C	Second	child	False
14	0	3	female	14.0	0	0	7.8542	S	Third	child	False
22	1	3	female	15.0	0	0	8.0292	Q	Third	child	False
27	0	1	male	19.0	3	2	263.0000	S	First	man	True
38	0	3	female	18.0	2	0	18.0000	S	Third	woman	False
...	...	...	...	...	...	...	...	...	...	...	...
853	1	1	female	16.0	0	1	39.4000	S	First	woman	False
855	1	3	female	18.0	0	1	9.3500	S	Third	woman	False
875	1	3	female	15.0	0	0	7.2250	C	Third	child	False
877	0	3	male	19.0	0	0	7.8958	S	Third	man	True
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False

102 rows × 15 columns

In [29]:

titanic[mask]

Out[29]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male
9	1	2	female	14.0	1	0	30.0708	C	Second	child	False
14	0	3	female	14.0	0	0	7.8542	S	Third	child	False
22	1	3	female	15.0	0	0	8.0292	Q	Third	child	False
27	0	1	male	19.0	3	2	263.0000	S	First	man	True
38	0	3	female	18.0	2	0	18.0000	S	Third	woman	False
...	...	...	...	...	...	...	...	...	...	...	...
853	1	1	female	16.0	0	1	39.4000	S	First	woman	False
855	1	3	female	18.0	0	1	9.3500	S	Third	woman	False
875	1	3	female	15.0	0	0	7.2250	C	Third	child	False
877	0	3	male	19.0	0	0	7.8958	S	Third	man	True
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False

102 rows × 15 columns

나이가 10세 미만이고 여성인 승객 추출

In [31]:

```
mask = (titanic.age < 10) & (titanic.sex == 'female')
titanic[mask]
titanic.loc[mask, :]
```

Out[31]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	de
10	1	3	female	4.00	1	1	16.7000	S	Third	child	False	
24	0	3	female	8.00	3	1	21.0750	S	Third	child	False	Na
43	1	2	female	3.00	1	2	41.5792	C	Second	child	False	Na
58	1	2	female	5.00	1	2	27.7500	S	Second	child	False	Na
119	0	3	female	2.00	4	2	31.2750	S	Third	child	False	Na
147	0	3	female	9.00	2	2	34.3750	S	Third	child	False	Na
172	1	3	female	1.00	1	1	11.1333	S	Third	child	False	Na
184	1	3	female	4.00	0	2	22.0250	S	Third	child	False	Na
205	0	3	female	2.00	0	1	10.4625	S	Third	child	False	
233	1	3	female	5.00	4	2	31.3875	S	Third	child	False	Na
237	1	2	female	8.00	0	2	26.2500	S	Second	child	False	Na
297	0	1	female	2.00	1	2	151.5500	S	First	child	False	
374	0	3	female	3.00	3	1	21.0750	S	Third	child	False	Na
381	1	3	female	1.00	0	2	15.7417	C	Third	child	False	Na
448	1	3	female	5.00	2	1	19.2583	C	Third	child	False	Na
469	1	3	female	0.75	2	1	19.2583	C	Third	child	False	Na
479	1	3	female	2.00	0	1	12.2875	S	Third	child	False	Na
530	1	2	female	2.00	1	1	26.0000	S	Second	child	False	Na
535	1	2	female	7.00	0	2	26.2500	S	Second	child	False	Na
541	0	3	female	9.00	4	2	31.2750	S	Third	child	False	Na
618	1	2	female	4.00	2	1	39.0000	S	Second	child	False	
634	0	3	female	9.00	3	2	27.9000	S	Third	child	False	Na
642	0	3	female	2.00	3	2	27.9000	S	Third	child	False	Na
644	1	3	female	0.75	2	1	19.2583	C	Third	child	False	Na
691	1	3	female	4.00	0	1	13.4167	C	Third	child	False	Na
720	1	2	female	6.00	0	1	33.0000	S	Second	child	False	Na
750	1	2	female	4.00	1	1	23.0000	S	Second	child	False	Na
777	1	3	female	5.00	0	0	12.4750	S	Third	child	False	Na
813	0	3	female	6.00	4	2	31.2750	S	Third	child	False	Na
852	0	3	female	9.00	1	1	15.2458	C	Third	child	False	Na



Out[31]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	de
10	1	3	female	4.00	1	1	16.7000	S	Third	child	False	
24	0	3	female	8.00	3	1	21.0750	S	Third	child	False	Na
43	1	2	female	3.00	1	2	41.5792	C	Second	child	False	Na
58	1	2	female	5.00	1	2	27.7500	S	Second	child	False	Na
119	0	3	female	2.00	4	2	31.2750	S	Third	child	False	Na
147	0	3	female	9.00	2	2	34.3750	S	Third	child	False	Na
172	1	3	female	1.00	1	1	11.1333	S	Third	child	False	Na
184	1	3	female	4.00	0	2	22.0250	S	Third	child	False	Na
205	0	3	female	2.00	0	1	10.4625	S	Third	child	False	
233	1	3	female	5.00	4	2	31.3875	S	Third	child	False	Na
237	1	2	female	8.00	0	2	26.2500	S	Second	child	False	Na
297	0	1	female	2.00	1	2	151.5500	S	First	child	False	
374	0	3	female	3.00	3	1	21.0750	S	Third	child	False	Na
381	1	3	female	1.00	0	2	15.7417	C	Third	child	False	Na
448	1	3	female	5.00	2	1	19.2583	C	Third	child	False	Na
469	1	3	female	0.75	2	1	19.2583	C	Third	child	False	Na
479	1	3	female	2.00	0	1	12.2875	S	Third	child	False	Na
530	1	2	female	2.00	1	1	26.0000	S	Second	child	False	Na
535	1	2	female	7.00	0	2	26.2500	S	Second	child	False	Na
541	0	3	female	9.00	4	2	31.2750	S	Third	child	False	Na
618	1	2	female	4.00	2	1	39.0000	S	Second	child	False	
634	0	3	female	9.00	3	2	27.9000	S	Third	child	False	Na
642	0	3	female	2.00	3	2	27.9000	S	Third	child	False	Na
644	1	3	female	0.75	2	1	19.2583	C	Third	child	False	Na
691	1	3	female	4.00	0	1	13.4167	C	Third	child	False	Na
720	1	2	female	6.00	0	1	33.0000	S	Second	child	False	Na
750	1	2	female	4.00	1	1	23.0000	S	Second	child	False	Na
777	1	3	female	5.00	0	0	12.4750	S	Third	child	False	Na
813	0	3	female	6.00	4	2	31.2750	S	Third	child	False	Na
852	0	3	female	9.00	1	1	15.2458	C	Third	child	False	Na

◀

▶

나이가 10세 미만 또는 60세 이상인 승객의 age, sex, alone 열 추출

In [32]:

```
mask = (titanic.age < 10) | (titanic.age >=60)
titanic.loc[mask, ['age', 'sex', 'alone']]
```

```
Out[32]:
```

	age	sex	alone
7	2.00	male	False
10	4.00	female	False
16	2.00	male	False
24	8.00	female	False
33	66.00	male	True
...	...	...	...
831	0.83	male	False
850	4.00	male	False
851	74.00	male	True
852	9.00	female	False
869	4.00	male	False

88 rows × 3 columns

참고. IPython Display 설정 변경

- https://pandas.pydata.org/docs/reference/api/pandas.get_option.html
- https://pandas.pydata.org/docs/reference/api/pandas.set_option.html

```
In [34]: # IPython 디스플레이 설정 값 가져오기 : 행과 열의 수
pd.get_option('display.max_columns')
pd.get_option('display.max_rows')
```

Out[34]: 20

Out[34]: 60

```
In [35]: # IPython 디스플레이 설정 변경: 출력할 최대 행/열의 수 지정
pd.set_option('display.max_columns', 10)
pd.set_option('display.max_rows', 10)

titanic.head()
```

```
Out[35]:
```

	survived	pclass	sex	age	sibsp	...	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	...	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	...	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	...	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	...	False	C	Southampton	yes	False
4	0	3	male	35.0	0	...	True	NaN	Southampton	no	True

5 rows × 15 columns

(2) isin() 메서드 활용

- isin() 메서드를 적용하면 특정 값을 가진 행들을 추출함
- 형식 : df.isin(추출값 리스트)

함께 탑승한 형제 또는 배우자 수가 3,4,5인 승객만 추출

- 불린 인덱싱 사용

```
In [37]: # mask = (titanic.sibsp == 3) | (titanic.sibsp == 4) | (titanic.sibsp == 5)
mask = (titanic.sibsp >= 3) & (titanic.sibsp <=5)

titanic.loc[mask]
```

```
Out[37]:
```

	survived	pclass	sex	age	sibsp	...	adult_male	deck	embark_town	alive	alone
7	0	3	male	2.0	3	...	False	NaN	Southampton	no	False
16	0	3	male	2.0	4	...	False	NaN	Queenstown	no	False
24	0	3	female	8.0	3	...	False	NaN	Southampton	no	False
27	0	1	male	19.0	3	...	True	C	Southampton	no	False
50	0	3	male	7.0	4	...	False	NaN	Southampton	no	False
...	...	...	...	...	...	...	...	...	...	...	...
787	0	3	male	8.0	4	...	False	NaN	Queenstown	no	False
813	0	3	female	6.0	4	...	False	NaN	Southampton	no	False
819	0	3	male	10.0	3	...	False	NaN	Southampton	no	False
824	0	3	male	2.0	4	...	False	NaN	Southampton	no	False
850	0	3	male	4.0	4	...	False	NaN	Southampton	no	False

39 rows × 15 columns

- isin() 메서드 사용

```
In [38]: mask = titanic.sibsp.isin([3,4,5])
titanic[mask]
```

Out[38]:

	survived	pclass	sex	age	sibsp	...	adult_male	deck	embark_town	alive	alone
7	0	3	male	2.0	3	...	False	NaN	Southampton	no	False
16	0	3	male	2.0	4	...	False	NaN	Queenstown	no	False
24	0	3	female	8.0	3	...	False	NaN	Southampton	no	False
27	0	1	male	19.0	3	...	True	C	Southampton	no	False
50	0	3	male	7.0	4	...	False	NaN	Southampton	no	False
...	...	...	...	...	...	...	...	...	...	...	...
787	0	3	male	8.0	4	...	False	NaN	Queenstown	no	False
813	0	3	female	6.0	4	...	False	NaN	Southampton	no	False
819	0	3	male	10.0	3	...	False	NaN	Southampton	no	False
824	0	3	male	2.0	4	...	False	NaN	Southampton	no	False
850	0	3	male	4.0	4	...	False	NaN	Southampton	no	False

39 rows × 15 columns

